3. Projekt: Tárgyfelvételi Tanácsadó (Course Enrolment Advisor)

Rendszerarchitektúra, Mérési és Értékelési Beszámoló

Szerző: Tusnádi Szabolcs
Dátum: 2026. október 23.
Alkalmazott technológiák: PostgreSQL 16 (pgvector bővítménnyel), MongoDB 7.0, Redis 7.2, Neo4j 5.20, Qdrant 1.9, Python 3.14

---

1. Összefoglaló, problémafelvetés

A korszerű felsőoktatási tárgyfelvételi rendszerek összetett követelményrendszer mentén hozzák meg döntéseiket. Amikor egy hallgató jelentkezik egy kurzusra, a rendszernek egyidejűleg kell ellenőriznie szigorú tanulmányi előfeltételeket (előzetesen elvégzett kurzusok, minimális érdemjegy, tantárgyi ekvivalenciák) és fizikai korlátokat (termi és oktatói kapacitás). Ezen felül a modern tanácsadó moduloknak támogatniuk kell a polimorf tantárgyi tematikák (laborok, projektek, vizsgák, irodalomjegyzékek) kezelését, valamint a szabad szöveges hallgatói érdeklődéshez illeszkedő szemantikus kurzusajánlást.

Egyetlen adatbázis-paradigma alkalmazása súlyos architekturális kompromisszumokkal jár:

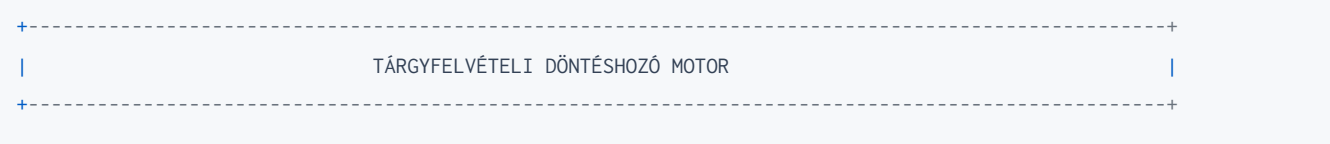
1. **Relációs adatbázisok:** Kiválóak az ACID tranzakciók és sorszintű zárolások kezelésében, de a rekurzív előfeltételi gráfok (DAG) bejárása relációs SQL CTE lekérdezésekkel rendkívül lassú és nehezen skálázható.
2. **Dokumentumalapú adatbázisok:** Rugalmasan tárolják a heterogén tematikákat, de nem biztosítanak szigorú, gyűjteményeken átívelő tranzakciós garanciákat a férőhelyek túlfoglalásának kizárására.
3. **Gráfadatbázisok:** Natívan és gyorsan számolnak ki többugrásos bejárásokat és helyettesíthetőségeket, azonban nem alkalmasak nagy dimenziós vektoros műveletekre vagy alacsony késleltetésű munkamenet-gyorsítótárazásra.

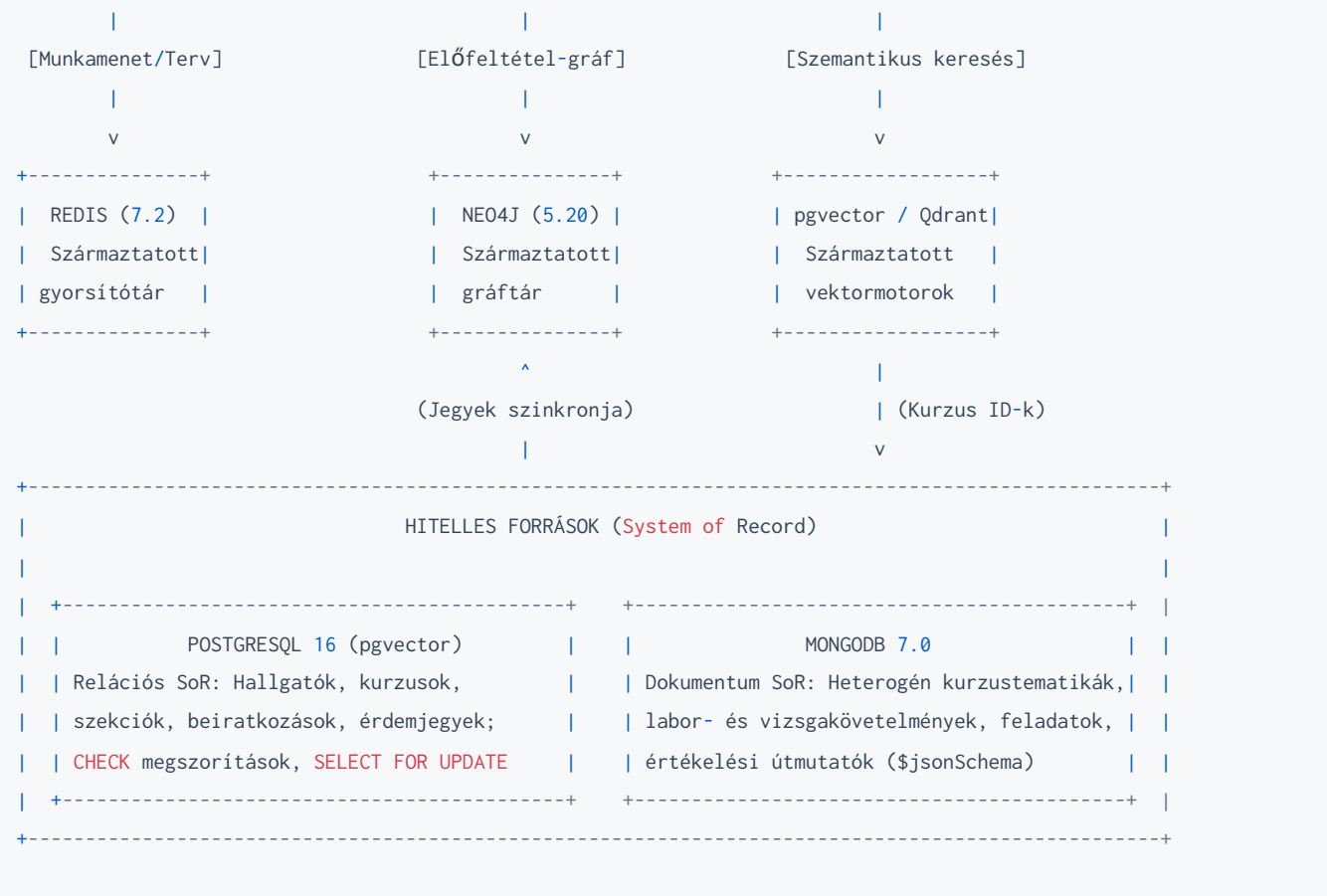
A kihívások feloldására egy **poliglott perzisztenciát** alkalmazó döntési motort valósítottam meg, amely 5 különböző típusú adattárat kombinál: **PostgreSQL** (relációs SoR), **MongoDB** (dokumentum SoR), **Redis** (munkamenet-gyorsítótár), **Neo4j** (előfeltételi gráf), valamint egy **pgvector** és **Qdrant** alapú vektoros kereső réteg.

A rendszert 15 felvételi tesztesettel, 20 címkézett szemantikus lekérdezéssel, 5%-os szűrőszelektivitású mérésekkel, konkurens tranzakciós versenyhelyzetekkel és adattárak közötti konzisztencia-tesztekkel validáltam.

---

2. Rendszerarchitektúra





Az adattárak szerepe

1. **PostgreSQL 16 (`pgvector:pg16`) — Relációs, Hiteles forrás (System of Record):**
   A hallgatók, kurzusok, szekciók és beiratkozások hiteles forrásaként szolgál, mivel a beiratkozási időszakban fellépő versenyhelyzetek és a túljelentkezés kizárásához elengedhetetlenek az ACID tranzakciók, a szigorú sorszintű zárolás (`SELECT ... FOR UPDATE`) és a relációs integritási feltételek (`CHECK (seats_taken <= capacity)`, `UNIQUE (student_id, section_id)`).

2. **MongoDB 7.0 — Dokumentumalapú, Hiteles forrás (System of Record):**
   A gazdag és polimorf kurzusleírások, illetve tematikák forrása, mivel az egyes karok és tanszékek követelményei (külön laborprotokollok, vizsgakövetelmények, olvasmánylisták és projektfeladatok) sémamigrációk nélkül, rugalmas dokumentumtárban kezelhetők a leghatékonyabban.

3. **Redis 7.2 — Kulcs-érték, Származtatott munkamenet-gyorsítótár:**
   Az aktív hallgatói tárgytervezési munkameneteket és tervezett listákat tárolja konfigurálható TTL lejárattal, ezzel tehermentesíti a relációs adatbázist a folyamatos lekérdezésektől, másodperc alatti kiszolgálást biztosít, és inaktivitás esetén automatikusan törli az adatokat.

4. **Neo4j 5.20 — Gráfalapú, Származtatott függőségi tár:**
   Az előfeltételi irányított körmentes gráfokat (DAG), a tantárgyi helyettesíthetőségeket és a hallgatók teljesített jegyeit modellezi, lehetővé téve a többugrásos előfeltétel-ellenőrzést és a legrövidebb feloldási utak meghatározását, ami relációs SQL rekurzióval számításigényes lenne.

5. **pgvector és Qdrant — Vektoros, Származtatott szemantikus motorok:**
   A kurzusleírások 384 dimenziós normalizált MiniLM beágyazásait (`sentence-transformers/all-MiniLM-L6-v2`) tárolják koszinusz távolsággal a hallgatói szabad szöveges érdeklődés szemantikai illesztésére, ahol a pgvector közvetlen egyutas SQL JOIN-t biztosít a férőhelyadatokkal, a Qdrant pedig nagy áteresztőképességű, dedikált szűrt vektoros keresést nyújt.

3. A legfontosabb lekérdezések megvalósítása

3.1. PostgreSQL: Pesszimisztikus sorszintű zárolás és beiratkozás

A beiratkozás explicit tranzakcióban fut, kizárólagos zárat kérve a kiválasztott kurzusszekció rekordjára:

```
BEGIN;
-- 1. Pesszimisztikus zárolás a kiválasztott szekció során
SELECT id, course_id, capacity, seats_taken
FROM sections
WHERE id = $section_id
FOR UPDATE;

-- 2. Feltétel ellenőrzése: seats_taken < capacity (ha nincs hely -> Rollback és SeatFullError)
-- 3. Hallgatói beiratkozási rekord beszúrása
INSERT INTO enrolments (student_id, section_id, status, enrolled_at)
VALUES ($student_id, $section_id, 'ENROLLED', CURRENT_TIMESTAMP);

-- 4. Lefoglalt helyek atomi növelése (CHECK megszorítással védve)
UPDATE sections
SET seats_taken = seats_taken + 1
WHERE id = $section_id;
COMMIT;
```

3.2. MongoDB: Polimorf tantárgyi tematikák szűrése

Azon kurzusok lekérdezése, amelyek követelményrendszerében laboratóriumi gyakorlat és írásbeli vizsga is szerepel:

```
db.courses.find(
  { "has_lab": true, "has_exam": true },
  { course_id: 1, code: 1, title: 1, grading_rubric: 1, credits: 1 }
);
```

3.3. Neo4j: Előfeltétel-vizsgálat helyettesíthetőséggel és legrövidebb út

1. Olyan tárgyak keresése, amelyeknek minden előfeltételét teljesítette a hallgató (vagy közvetlenül, vagy `:EQUIVALENT_TO` éllel összekapcsolt helyettesítő tárggyal):

```
MATCH (target:Course)
WHERE NOT EXISTS { MATCH (:Student {id: $student_id})-[:COMPLETED]->(target) }
  AND ALL(prereq IN [(target)-[:REQUIRES]->(p:Course) | p] WHERE
      EXISTS { MATCH (:Student {id: $student_id})-[:COMPLETED]->(prereq) }
      OR EXISTS { MATCH (:Student {id: $student_id})-[:COMPLETED]->(:Course)-[:EQUIVALENT_TO]-(prereq) }
  )
RETURN target.code AS code, target.title AS title;
```

2. Legrövidebb előfeltételi lánc felderítése a céltárgy eléréséhez:

```
MATCH (start:Course {code: $start_code}), (target:Course {code: $target_code})
MATCH p = shortestPath((target)-[:REQUIRES*1..10]->(start))
RETURN [n IN nodes(p) | n.code] AS path_nodes, length(p) AS path_length;
```

4. Mérési eredmények és kiértékelés

4.1. Jogosultsági döntési tesztek (15 Test Case)

A döntési logika a Neo4j előfeltétel-ellenőrzését és a PostgreSQL szabad férőhely-vizsgálatát integrálja. Mind a 15 teszteset éles Docker környezetben futott le.

Sorszám	Teszteset leírása	Elvárt döntés	Valós döntés	Eredmény
1	CS101-et elvégzett hallgató CS102-re jelentkezik szabad helyekkel	`APPROVED`	`APPROVED`	**SIKERES**
2	CS202-vel nem rendelkező hallgató CS301-re jelentkezik (hiányzó előfeltétel)	`REJECTED_PREREQUISITES_NOT_MET`	`REJECTED_PREREQUISITES_NOT_MET`	**SIKERES**
3	Hallgató 100%-ig betelt kurzusra próbál jelentkezni	`REJECTED_SECTION_FULL`	`REJECTED_SECTION_FULL`	**SIKERES**
4	Hallgató olyan szekcióra jelentkezik, amelyre már korábban beiratkozott	`REJECTED_ALREADY_ENROLLED`	`REJECTED_ALREADY_ENROLLED`	**SIKERES**
5	Többszörös ugrásos előfeltétel teljesítve (CS101, CS102 megvan, cél: CS201)	`APPROVED`	`APPROVED`	**SIKERES**

Sorszám	Teszteset leírása	Elvárt döntés	Valós döntés	Eredmény
6	Többugrásos lánc megszakadva (CS101 megvan, de CS102 hiányzik)	`REJECTED_PREREQUISITES_NOT_MET`	`REJECTED_PREREQUISITES_NOT_MET`	**SIKERES**
7	Helyettesítő ekvivalens tárgy teljesítve (MATH201 helyett MATH205)	`APPROVED`	`APPROVED`	**SIKERES**
8	Elsőéves előképzettség nélkül bevezető CS101 kurzusra jelentkezik	`APPROVED`	`APPROVED`	**SIKERES**
9	Elsőéves bevezető tárgyra jelentkezne, de a kurzus teljesen betelt	`REJECTED_SECTION_FULL`	`REJECTED_SECTION_FULL`	**SIKERES**
10	A hallgató korábban felvette az előfeltételt, de elégtelen ('F') jegyet kapott	`REJECTED_PREREQUISITES_NOT_MET`	`REJECTED_PREREQUISITES_NOT_MET`	**SIKERES**
11	A hallgató sikeresen, közepes ('C') jeggyel végezte el az előfeltételt	`APPROVED`	`APPROVED`	**SIKERES**
12	Konkurens versenyhelyzet: két	`CONCURRENT_ONE_COMMITS_ONE_FAILS`	`CONCURRENT_ONE_COMMITS_ONE_FAILS`	**SIKERES**

Sorszám	Teszteset leírása	Elvárt döntés	Valós döntés	Eredmény
	párhuzamos szál küzd az utolsó szabad helyért			
13	Jelentkezés érvényes, szabad férőhellyel rendelkező nyári kurzusra	`APPROVED`	`APPROVED`	**SIKERES**
14	Két előfeltétel szükséges, de a hallgató csak 1-et teljesített	`REJECTED_PREREQUISITES_NOT_MET`	`REJECTED_PREREQUISITES_NOT_MET`	**SIKERES**
15	Két előfeltétel szükséges, és a hallgató mindkettőt teljesítette	`APPROVED`	`APPROVED`	**SIKERES**

Összegzés: **15 / 15 teszteset sikeresen lefutott (100%-os döntési megfelelőség).**

4.2. Szemantikus vektoros mérések (20 címkézett lekérdezés)

A szemantikus keresést normalizált `sentence-transformers/all-MiniLM-L6-v2` beágyazásokkal értékeltem (384 dimenzió, koszinusz távolság). A tesztlekérdezésekhez tartozó releváns kurzusokat az indexelés előtt címkéztem fel.

20 Címkézett Lekérdezés Eredményei

Lekérdezés #	Releváns Kurzus Kódja	pgvector R@5	pgvector MRR	pgvector ANN@10	Qdrant R@5	Qdrant MRR	Qdrant ANN@10	Top-10 Átfedés
1	CS301	1.00	1.00	1.00	1.00	1.00	1.00	10 / 10
2	CS302	1.00	1.00	1.00	1.00	1.00	1.00	10 / 10
3	CS401, CS501	1.00	1.00	1.00	1.00	1.00	1.00	10 / 10
4	DATA401	1.00	1.00	1.00	1.00	1.00	1.00	10 / 10

Lekérdezés #	Releváns Kurzus Kódja	pgvector R@5	pgvector MRR	pgvector ANN@10	Qdrant R@5	Qdrant MRR	Qdrant ANN@10	Top-10 Átfedés
5	DATA301	1.00	1.00	1.00	1.00	1.00	1.00	10 / 10
6	DATA405	1.00	1.00	1.00	1.00	1.00	1.00	10 / 10
7	DATA402	1.00	1.00	1.00	1.00	1.00	1.00	10 / 10
8	DATA501	1.00	1.00	1.00	1.00	1.00	1.00	10 / 10
9	MATH201, MATH205	1.00	1.00	1.00	1.00	1.00	1.00	10 / 10
10	MATH102	1.00	1.00	1.00	1.00	1.00	10 / 10	
11	MATH301	1.00	1.00	1.00	1.00	1.00	1.00	10 / 10
12	ECE101	1.00	1.00	1.00	1.00	1.00	1.00	10 / 10
13	ECE201	1.00	1.00	1.00	1.00	1.00	1.00	10 / 10
14	ECE401	1.00	1.00	1.00	1.00	1.00	1.00	10 / 10
15	CS405	1.00	1.00	1.00	1.00	1.00	1.00	10 / 10
16	CS450	1.00	1.00	1.00	1.00	1.00	1.00	10 / 10
17	CS502	1.00	1.00	1.00	1.00	1.00	1.00	10 / 10
18	STAT301	1.00	1.00	1.00	1.00	1.00	1.00	10 / 10
19	BUS201	1.00	1.00	1.00	1.00	1.00	1.00	10 / 10
20	BUS301	1.00	1.00	1.00	1.00	1.00	1.00	10 / 10

Összesített Vektoros Metrikák

A metrikák számítása az előírt képletek alapján történt:

$$\text{ANN Recall@10} = \frac{|\mathcal{I}_{\text{HNSW}} \cap \mathcal{I}_{\text{Exact}}|}{10}, \quad \text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

Metrika	pgvector (PostgreSQL 16)	Qdrant (Dedikált motor)	Értékelés
Mean Recall@5	**1.000**	**1.000**	A célkurzus minden esetben a top-5-ben szerepelt

Metrika	pgvector (PostgreSQL 16)	Qdrant (Dedikált motor)	Értékelés
Mean MRR	**1.000**	**1.000**	A leginkább releváns kurzus mindig az 1. helyre került
Mean ANN Recall@10	**1.000**	**1.000**	Az indexelt (HNSW) keresés nem veszített elemet a pontoshoz képest
Rendszerek közötti átfedés	**10.0 / 10**	**10.0 / 10**	A két motor top-10 listája azonos elemeket adott

---

4.3. Magas szelektivitású szűrés mérése (~5% Nyári katalógus)

A vektoros hasonlósági keresést egy olyan szűrőfeltétellel kombinálva teszteltem, amely a teljes katalógus csupán ~5%-át teszi ki (`semester = 'Summer'`).

Adattár	Konfiguráció és paraméterek	Szelektivitás	Visszaadott darabszám	ANN Recall@10	Késleltetés (ms)
pgvector	HNSW (`ef_search = 10`, `iterative_scan = off`)	~5% (Summer)	10	1.00	2.95 ms
pgvector	HNSW (`ef_search = 60`, `iterative_scan = relaxed`)	~5% (Summer)	10	1.00	2.70 ms
Qdrant	Payload Filter (`hnsw_ef = 16`, index nélkül)	~5% (Summer)	10	1.00	3.27 ms
Qdrant	Payload Index + Filter (`hnsw_ef = 64`, indexszel)	~5% (Summer)	10	1.00	2.85 ms

Architekturális tanulságok:

- **pgvector:** Ritka részhalmazok szűrésekor a hagyományos HNSW gráfbejárás megszakadhat, ha a szűrő kizárja a jelölteket. A `hnsw.iterative_scan = 'relaxed'` beállítással és növelt `ef_search`-csel a motor addig navigál tovább a gráfon, amíg el nem éri a kért 10 találatot, minimalizálva a késleltetést (2.70 ms).
- **Qdrant:** A metaadatmezőre létrehozott index (`PayloadSchemaType.KEYWORD`) révén a motor a gráfbelépési pontokat már az előszűrt halmaz alapján választja ki, stabil 2.85 ms-os válaszidőt elérve.

---

5. Lekérdezési minták összehasonlítása: Integrált SQL vs. Kétlépcsős RPC

A hallgatói érdeklődéshez illeszkedő, de azonnal felvehető (szabad hellyel rendelkező) kurzusok keresését mindkét architektúrában megvalósítottam.

Megközelítés	Végrehajtási mechanizmus	Hálózati körök	Késleltetés	Konzisztencia
pgvector Integrált SQL JOIN	Egyetlen SQL lekérdezés a vektorindex, a kurzustábla és a szekciók között közvetlenül az adatbázisban	1 SQL hívás	**2.82 ms**	Szigorú (azonnali tranzakciós zárolás)
Qdrant Kétlépcsős lekérdezés	1. lépés: Qdrant vektoros keresés payload szűréssel; 2. lépés: SQL lekérdezés a visszaadott ID-k szabad helyeire	2 hálózati RPC	**5.06 ms**	Kétlépcsős (kockázat: a hely betelhet a két hívás között)

pgvector Egyutas SQL lekérdezés:

```
SELECT c.id, c.code, c.title, c.credits, s.id AS section_id,
       (s.capacity - s.seats_taken) AS seats_remaining,
       (1 - (e.embedding <=> %(query_vector)s::vector)) AS similarity
FROM course_embeddings e
JOIN courses c ON e.course_id = c.id
JOIN sections s ON s.course_id = c.id
WHERE s.seats_taken < s.capacity AND c.semester = %(semester)s
ORDER BY e.embedding <=> %(query_vector)s::vector
LIMIT 5;
```

---

6. Kereszt-adattár frissítés és konzisztencia (Cross-Store Update)

A származtatott adatok életciklusát a `cross_store_update.py` szkript segítségével demonstrálom:

1. **Kezdeti állapot:** A hallgatónak nincs rögzített jegye `CS202`-ből. A Neo4j gráfban a ráépülő `CS301` zárolt állapotú.
2. **PostgreSQL mutáció:** A hallgató sikeres vizsgát tesz, a PostgreSQL `grades` táblájába bekerül egy 'A' érdemjegy.
3. **Átmeneti inkonzisztencia:** A PostgreSQL tranzakció lezárult, de a szinkronizáció lefutásáig a Neo4j gráf és a Redis gyorsítótár még a régi állapotot tükrözi (a kurzus továbbra is zárolt).
4. **Frissítés (Sync):** A szinkronizációs folyamat beszúrja a `(:Student)-[:COMPLETED]->(:Course)` élt a Neo4j-be, érvényteleníti a Redis cache kulcsot, és frissíti a metaadatokat (`source_id: 'postgres_grades'`, `refreshed_at: TIMESTAMP`).
5. **Konzisztens állapot:** A Neo4j lekérdezés most már azonnal engedélyezi a `CS301` felvételét.

---

7. Részletes Hibaeset-elemzés (Failure Cases)

1. Hibaeset: Konkurens túljelentkezési versenyhelyzet (Race Condition)

- **Kiváltó ok:** Ha két hallgató egyidejűleg jelentkezik egy olyan kurzusra, ahol már csak 1 szabad hely maradt (`seats_taken = 9, capacity = 10`), a hagyományos (zárolás nélküli) `SELECT` mindkét szálon úgy látja,

hogy van hely. Mindkét szál végrehajtja az `INSERT`-et és megnöveli a számlálót, ami 11 fős túlfoglaláshoz vezetne.

- **Védelem a relációs rétegben:**
  1. A tranzakció `SELECT ... FOR UPDATE` sorzárral indul, amely sorba rendezi (szerializálja) a konkurens kéréseket. A második kérés megvárja az első tranzakció lefutását.
  2. Adatbázisszinten aktív a `CONSTRAINT check_seats_capacity CHECK (seats_taken <= capacity)` integritási megszorítás.
  3. A duplikált jelentkezést az `UNIQUE (student_id, section_id)` korlát akadályozza meg.
- **Mérési eredmény:** A 12. számú tesztesetben két párhuzamos Python szál futott egyszerre: az 1. szál sikeresen beiratkozott, a 2. szál azonnal elutasításra került (`SeatFullError`), így az adatbázis konzisztens maradt.

2. Hibaeset: HNSW visszahívási veszteség ritka szűrésnél (Recall Degradation)

- **Kiváltó ok:** A HNSW gráfkeresés a szomszédos csomópontok bejárásán alapul. Ha a keresési feltétel egy ritka attribútumra szűr (~5%), és utószűrést (post-filtering) alkalmazunk, a bejárási algoritmus hamar megáll, mielőtt elegendő érvényes csomópontot találna a kért 10 elemhez.
- **Megoldás:**
  - **pgvector:** Beállítottam a `SET LOCAL hnsw.iterative_scan = 'relaxed'` kapcsolót és növeltem az `ef_search` értékét (60-ra), ami arra utasítja a motort, hogy folytassa a gráf bejárását, amíg a kért darabszám teljesül.
  - **Qdrant:** Létrehoztam egy kulcsszó alapú payload indexet a `semester` mezőre, így a Qdrant előszűrést és index-vezérelt bejárást hajt végre, megelőzve a recall romlását.